

Getting Started

Development on the main AzuraCast application should always be applicable to a broad community of radio station operators and not specific features to one station or group of stations.

If you would like to build a set of features specific to one station or group of stations, you should take advantage of AzuraCast's plugin architecture. The plugin system takes advantage of event listeners that are built into AzuraCast itself. Check out the [example plugin](#)  for more details on what is possible via plugins.

Contributing Code Changes

AzuraCast is open-source software, and as part of this dedication to openness and transparency, we fully support contributions from members of the community who are skilled in the languages that we use to build our applications.

A majority of our repositories come with an `.editorconfig` file in the root, which will set many standards for indentation, punctuation and other style items for you automatically. You may need to enable EditorConfig support in your IDE of choice.

If your IDE does not support EditorConfig, the most important standard to remember that we follow is the PHP Framework Interoperability Group's [PSR-12 Extended Coding Style](#)  standard.

Accessibility, security, and modern best practices are very important in AzuraCast's development. Any newly contributed code can, and should, take advantage of the full suite of new features made available in PHP 8.0 and newer.

Contributions are also welcome in the supporting technologies used to make AzuraCast possible, such as:

- Dockerfiles (see [our separate repositories](#)  for Docker containers)
- [Ansible configuration](#)  for Ansible installs

Setting Up a Local Environment

Regardless of your host operating system, it's highly recommended to use **Docker** when developing locally, as it offers portability, convenience, and a very close approximation of how AzuraCast runs on production environments.

For the steps below, we're assuming you've created a folder where you will store all of your AzuraCast-related projects, like `/home/myuser/azuracast/`.

You will need Git and Docker installed locally. If you're on Windows or Mac, the best way to use Docker is via the native [Docker Desktop](#)  applications for those platforms.

For Windows, an installer tool like [Scoop](#)  is highly recommended for dependencies like Git and your terminal of choice. A third-party shell client like Cmder is also often helpful.

Clone the Core Repository

Using Git, clone the AzuraCast core repository into a subfolder of your working directory.

Note for Windows developers: Before cloning the repositories, you should ensure your Git is locally configured to not automatically convert line endings from Linux style (LF) to Windows style (CRLF), which will break AzuraCast. You can set this globally by running:

```
git config --global core.autocrlf input
```

To do this, your initial command should look like:

```
git clone https://github.com/AzuraCast/AzuraCast.git
```

Setup via Docker Utility Script

We have added a developer-specific helper tool to our Docker Utility Script that takes care of a lot of the common setup tasks for you. This script will copy the default environment configuration files, clone the additional repositories, and even install Docker and Docker Compose for you if they aren't installed already.

From the previous command, you can run this script by running:

```
cd AzuraCast
bash docker.sh install-dev
# Alternatively, you can use `make install` if `make` is installed on your
system
```

Modify the Environment File

During the developer setup process, if you haven't done so already, you will be prompted to customize your `azuracast.env` file to make sure the proper values are input into this file.

Many of the values left blank by default are what are called "fixtures", which will automatically populate parts of the database upon initial installation, saving you the

trouble of setting those values up every time you reset your database or code.

If you want to populate those values, you can modify them in the `azuracast.env` file, where they will look like the fields below:

```
INIT_BASE_URL=http://azuracast.local
INIT_INSTANCE_NAME="local development"
INIT_ADMIN_EMAIL=
INIT_ADMIN_PASSWORD=
... etc.
```

When initializing a development instance, AzuraCast will create both a standard demo account and system admin user profile, using the values set in `azuracast.env` if provided. `INIT_DEMO_API_KEY` and `INIT_ADMIN_API_KEY` are associated with the demo and admin users respectively, and should be a valid split-token key. The default login credentials for the demo user are `demo@azuracast.com` with password `demo`. Additionally, AzuraCast will create a demo station AzuraTest Radio (`azuratest_radio`) that will use the defined `INIT_MUSIC_PATH` and `INIT_PODCASTS_PATH`.

If you would prefer to follow the traditional post-installation setup process instead, just remove any fields from the `azuracast.env` file that start with `INIT_`.

Following these steps, re-run the prior command (`bash docker.sh install-dev` or `make install`) to build the Docker image and run the containers. After the script finishes (as indicated by `✓ Container azuracast Started`), you may need to wait a few more minutes for the initialization process to finish running. You can check the progress by running `docker logs -f azuracast`.

Common Tasks

Many common tasks are made simpler by the addition of a `Makefile` to our core repository. It's strongly recommended to install `make` on your host OS or inside WSL.

Rebuilding the Docker Images

By default, developer installations bind mount the current code from your local computer into the Docker container, so changes to the code will be visible immediately in the running web application.

If you need to make changes to the containers themselves, though, or your containers get out of date with the ones in our repositories, you can rebuild them by running:

```
docker-compose build  
# OR  
make build
```

You should then restart your installation by running:

```
docker-compose down  
docker-compose up -d  
# OR  
make restart
```

If you want to fully rebuild, restart, and re-run the setup process after a Docker image change, you can run all of those as one command:

```
make update
```

Building Static Assets

AzuraCast automatically builds static assets as they're changed in the dev environment. Modifications inside the `/frontend` folder are automatically watched and rebuilt when developing, then compiled down into static files as part of our Docker image build process.

Accessing bash Inside the Container

It is common to need to access the bash shell inside the running web container in order to see or modify the current state of the application while it's running.

There is a helper command you can run using the Docker Utility Script to easily accomplish this:

```
bash docker.sh bash  
# OR  
make bash
```

Customize Local SSL

Out of the box, AzuraCast comes with a self-signed SSL certificate that you can use for local development. Most browsers will complain about this, but you can add an exception to allow you to proceed anyway.

If you want to use a valid SSL certificate on your local installation, we highly recommend a tool called [mkcert](#) [↗](#).

We commonly use this tool to register a local SSL-enabled domain at `azuracast.local`. You will need to edit your `/etc/hosts` file to point this domain to your own computer as a one-time change.

The process for installing this certificate into the AzuraCast ecosystem looks like this command, run from the main repository's project root:

```
mkcert -cert-file util/local_ssl/azuracast.local.crt -key-file  
util/local_ssl/azuracast.local.key azuracast.local
```

To make your new certificate take effect, you will need to restart the Docker containers using:

```
docker-compose down  
docker-compose up -d  
# OR  
make restart
```

Translations (Locales)

Locales are managed by the application in two places: the backend PHP code and the frontend (primarily Vue.js) JavaScript code. When new locales are added or translations are changed, they should be processed in both locations.

There are two steps to the translation process:

Generating New Translations

When new strings have been added to the application, they should be added to the .POT (localization template) file, so that our Crowdin translation service recognizes it as a translatable string.



```
bash docker.sh cli locale:generate
```

```
# OR
```

```
make generate-locales
```

Import New Translated Strings

When strings have been translated, they should be converted back into optimized files that can easily be read by the PHP and Vue.js parts of the application, respectively.



```
bash docker.sh cli locale:import
```

```
# OR
```

```
make import-locales
```